

Distributed Query Processing

1 Query Cost Criteria

In centralized systems, the primary cost metric is **disk accesses**. In distributed systems, additional factors must be considered:

- **Data Transmission:** The cost of moving data over the network.
- **Parallelism:** Gain in performance from processing different parts of a query across multiple sites simultaneously.

2 Query Transformation

Queries on global relations must be translated into algebraic queries on fragments. Relation r is replaced by the expression used to reconstruct it.

Optimization Example

Consider horizontal fragmentation of `account` into `account1` (Hillside) and `account2` (Valleyview). The query $\sigma_{branch_name='Hillside'}(account)$ becomes:

1. **Initial:** $\sigma_{branch_name='Hillside'}(account_1 \cup account_2)$
2. **Transformed:** $\sigma_{branch_name='Hillside'}(account_1) \cup \sigma_{branch_name='Hillside'}(account_2)$
3. **Optimized:** Since `account1` contains only Hillside tuples, the selection is redundant. Since `account2` contains only Valleyview tuples, the selection result is $\emptyset$.
4. **Final:** Return `account1` directly.

3 Join Processing Strategies

Suppose we Join three relations (`account` $\bowtie$ `depositor` $\bowtie$ `branch`) stored at site S_1, S_2, S_3 respectively, and the result is needed at S_I .

Shipping Strategies

- **Centralized Processing:** Ship all copies to S_I and compute locally.
- **Iterative Shipping:** Ship *account* to S_2 , compute *account* $\bowtie$ *depositor*, ship result to S_3 , etc.

Decision Factors: Amount of data, transmission cost per block, and local processing speeds.

4 Semijoin Strategy

Used to reduce the amount of data shipped between sites during a join $r_1 \bowtie r_2$ (where r_1 is at S_1 and r_2 is at S_2).

Semijoin Steps (Result at S_1)

1. Compute $temp_1 \leftarrow \Pi_{R_1 \cap R_2}(r_1)$ at S_1 .
2. Ship $temp_1$ from S_1 to S_2 .
3. Compute $temp_2 \leftarrow r_2 \bowtie temp_1$ at S_2 .
4. Ship $temp_2$ (only relevant r_2 tuples) from S_2 to S_1 .
5. Compute $r_1 \bowtie temp_2$ at S_1 .

Formal Definition: The semijoin $r_1 \bowtie r_2$ is defined as $\Pi_{R_1}(r_1 \bowtie r_2)$, selecting only the tuples of r_1 that have a match in r_2 .

5 Exploiting Parallelism

Parallelism can be achieved by coordinating joins across multiple pairs of sites.

Parallel Execution: $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$

- Site S_1 and S_2 compute $r_1 \bowtie r_2$ in parallel with S_3 and S_4 computing $r_3 \bowtie r_4$.
- Intermediate tuples are piped toward the final site (e.g., S_1) as they are produced, allowing the final join to start even before the first-level joins finish.

Efficient distributed query processing relies on minimizing transmission costs and maximizing the use of parallel resources across the network.